

UD1-Introducción a las BBDDs

1. INTRODUCCIÓN

2. DATOS E INFORMACIÓN

3. SISTEMA DE BASE DE DATOS

3.1. Componentes

3.2. Base de datos

3.2.1. Características de una BD

3.2.2. Supuesto de una BD

3.2.3. Metadatos de una BD

3.2.4. Niveles de abstracción en una BD

3.3. SGDB (en inglés, DBMS DataBase Management System)

3.3.1. Objetivos de los SGDB

3.3.2. Elección del SGDB

3.4. SQL

4. HISTORIA DE LAS BASES DE DATOS

4.1. Década de los años 50: Ficheros

4.2. Década de los años 60: BD Jerárquicas y en Red

4.3. Década de los años 70: Modelo Relacional y DBMS

4.3.1. Años 1970-72

4.3.2. Año 1976

4.3.2. Segunda mitad de los años 70

4.4. Década de los años 80

4.5. Década de los años 90

4.6. Actualidad

4.6.1. Panorama actual de las bases de datos NoSQL

5. MODELOS DE DATOS COMO INSTRUMENTO DE DISEÑO DE BASES DE DATOS

[5.1. ¿Qué es un modelo de datos?](#)

[5.2. Modelos de datos más utilizados](#)

[5.3. Modelo Relacional vs NoSQL](#)

[6. USUARIOS DE BD](#)

[6.1. Tareas de los DBAs](#)

[7. BASES DE DATOS DISTRIBUIDAS](#)

[7.1. Ventajas de los SGBDD](#)

[7.2. Inconvenientes de los SGBDD](#)

[7.3. CAP Theorem](#)

[7.4. Supuesto de aplicación distribuida](#)

[7.5. Técnicas para la creación de BD distribuidas](#)

[8. BIG DATA](#)

[8.1. Conceptos básicos](#)

[8.2. Fases del ciclo de vida](#)

[8.3. Casos de uso](#)

1. INTRODUCCIÓN

En el mundo actual existe una cada vez mayor demanda de datos. Esta demanda siempre ha sido patente en empresas y sociedades, pero en este siglo XXI la demanda se ha disparado más debido al acceso multitudinario a las redes en Internet por parte de usuarios finales como de múltiples y diversos dispositivos (IoT, Internet of Things).

Antes de la aparición de las aplicaciones informáticas, las empresas tenían como únicas herramientas de gestión de datos los cajones, carpetas y fichas en las que se almacenaban los datos. En ese proceso manual, el tiempo requerido para manipular estos datos era enorme. Sin embargo, el proceso de aprendizaje era sencillo puesto que se usaban elementos muy familiares para el usuario.

Por ello, la informática ha adaptado sus herramientas para que los elementos que el usuario maneja en el ordenador se parezcan a los que utilizaba manualmente. Así, en informática se sigue hablando de archivos, formularios, carpetas, directorios, etc.

Ahora empezaremos por ver los conceptos básicos de las bases de datos, relacionándolos con los sistemas de gestión de archivos, para después empezar a hablar de los sistemas gestores de bases de datos y sus tipos.

2. DATOS E INFORMACIÓN

En primer lugar vamos a establecer una diferencia entre dos términos básicos que suelen utilizarse con bastante frecuencia en entornos de bases de datos y sistemas de información. Se trata de la diferencia entre **datos e información**.

Al hablar de datos nos referimos a hechos aislados, como por ejemplo, Juan tiene una cuenta corriente en Abanca. Por el contrario, al hablar de información nos estamos refiriendo a datos procesados, organizados o resumidos. Por ejemplo, la respuesta a la pregunta ¿Cuál es el saldo de las cuentas corrientes de Juan? ¿Cuál ha sido el gasto mensual medio de Juan en los últimos 6 meses en alimentación? Esto sería información.

Los datos suelen ser magnitudes numéricas o captadas, pero también pueden ser nombres o conjuntos de símbolos; o valores cualitativos; o frases enteras; o imágenes, sonidos, colores, olores.

Los datos no tienen sentido por sí mismos, pero al ser procesados y

contextualizados se convierten en información cierta y disponible para conocer un fenómeno, tomar decisiones o ejecutar acciones. Los datos por sí mismos no permiten la adopción de la decisión más conveniente por no aportar los conocimientos necesarios.

Solo una elaboración adecuada de los datos (un procesado de los datos) nos proporcionará el conocimiento deseado. La información es, pues, el resultado de esta transformación, de este **PROCESADO DE LOS DATOS**.

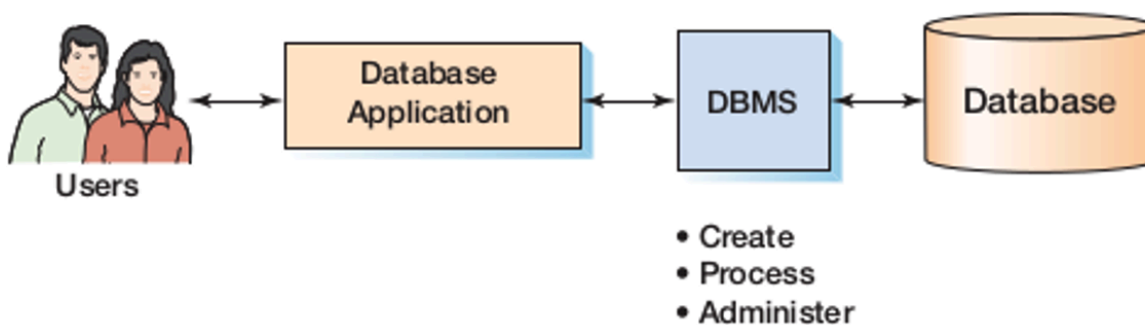
Por tanto, en el mundo empresarial, la información es un recurso vital, por lo que las empresas siempre han hecho uso de los medios que estaban a su alcance para el procesamiento de datos y la gestión de información. Entre estos medios se encuentran los Sistemas de Información que organizan los datos para producir información.

El desarrollo de los Sistemas de Bases de Datos se ha convertido en crucial para los Sistemas de Información, puesto que son su núcleo: son los que le proporcionan los datos que necesitan para la elaboración de la información.

Además, con la reciente explosión de la IA, los datos son el pilar fundamental para entrenar modelos de machine learning que consigan hacer predicciones futuras sobre datos pasados. De ahí a que los datacenters se hayan expandido exponencialmente en el mundo globalizado. [Video](#)

3. SISTEMA DE BASE DE DATOS

3.1. Componentes

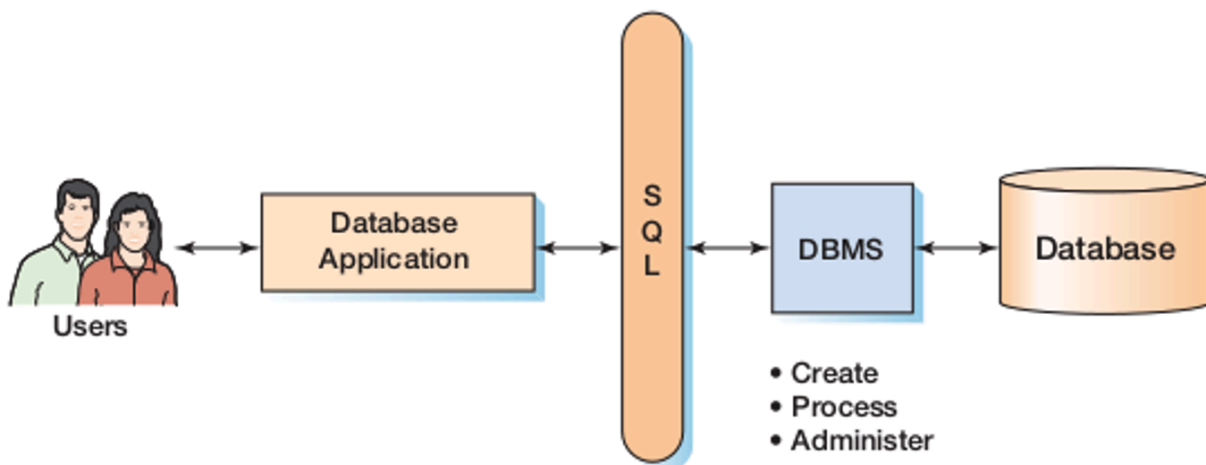


Típicamente consta de cuatro componentes:

- **Usuarios:** son los que utilizan la base de datos. Hacen uso de formularios para leer, entrar y consultar los datos y producen informes con la consecuente información.

- **Aplicación de acceso a la BD:** conjunto de uno o más programas que sirven como intermediario entre el usuario y el SGBD. Leen o modifican la base de datos mediante el envío de peticiones SQL al SGBD. También presentan al usuario los formularios e informes. Estos programas pueden ser adquiridos en compañías de software, pero normalmente están escritos manualmente. Las aplicaciones no acceden directamente a los datos, sino que le piden al SGBD las operaciones que quieren realizar.
- **Sistema Gestor de la Base de Datos (SGBD, en inglés DBMS, DataBase Management System):** conjunto de programas o librerías utilizados para crear, procesar y administrar la base de datos. Recibe las peticiones codificadas en SQL y las traduce en acciones sobre la base de datos. Debido a su gran complejidad, las empresas nunca desarrollan su propio SGBD, sino que adquieren las licencias para hacer uso de uno existente (Ver Apartado 3.3).
- **Base de datos:** colección de tablas relacionadas y otras estructuras (Ver Apartado 3.2).

Añadir, que dada la importancia de **SQL** (Structured Query Language), reconocido como estándar internacional comprensible por muchos de los SGBD en el procesamiento de la base de datos y el hecho de que las aplicaciones de acceso típicamente envían sentencias SQL al SGBD para su procesamiento, puede considerarse como un quinto componente del sistema (Ver Apartado 3.4).



3.2. Base de datos

Una **Base de Datos (BD)** es una **colección o repositorio de datos integrados, almacenados en soporte secundario (no volátil) y con redundancia controlada**. Los datos, que deben ser compartidos por diferentes usuarios y aplicaciones, deben mantenerse independientes de ellos y su definición (estructura de la BD), única y almacenada junto con los

datos, debe apoyarse en un modelo de datos, el cual debe permitir captar las interrelaciones y restricciones existentes en el mundo real. Los procedimientos de actualización y recuperación, comunes y bien determinados, facilitarán la seguridad del conjunto de los datos", De Miguel et al. (1999).

Veamos en qué consiste cada uno de los aspectos citados en esta definición de Base de datos.

La BD es un conjunto de datos relativos a una determinada parcela del mundo real (por ejemplo, una biblioteca, una empresa petroquímica, una universidad, etc.) que se almacenan en un soporte informático no volátil (es decir, dispositivos de memoria secundaria como discos, cintas, etc., que hacen que los datos no desaparezcan "cuando no se están usando").

Además, no debe existir redundancia, es decir, no deben existir duplicidades perjudiciales ni innecesarias (si puede ser un determinado tipo de dato, por ejemplo, los datos de un cliente de una empresa, sólo deben aparecer en un lugar en la BD). En ocasiones, es necesaria cierta redundancia (a nivel de almacenamiento físico, por ahora diremos que el nivel físico concierne a cómo se almacenan los datos en los archivos de la BD) que mejora la eficiencia de la BD, por ejemplo, frente a determinados tipos de consultas de datos. Sin embargo, esta redundancia siempre debe ser controlada por el sistema para que no se produzcan inconsistencias; piensa qué sucedería si los datos de los clientes de una empresa se repitiesen en varias partes de la BD y no se controlara: puede ocurrir que si un cliente cambia de dirección postal y sólo se actualiza esta información en uno de los sitios, entonces la BD quedaría en estado inconsistente (el cliente aparece con datos distintos en distintas partes de la BD).

Por otra parte, las BD deben atender a múltiples usuarios de la organización (informáticos que desarrollan programas de acceso a la BD, administrativos usuarios de las aplicaciones, etc.) así como a diferentes aplicaciones (por ejemplo, aplicaciones de contabilidad, de facturación, etc., todas ellas accediendo a los datos contenidos en la BD de la empresa).

3.2.1. Características de una BD

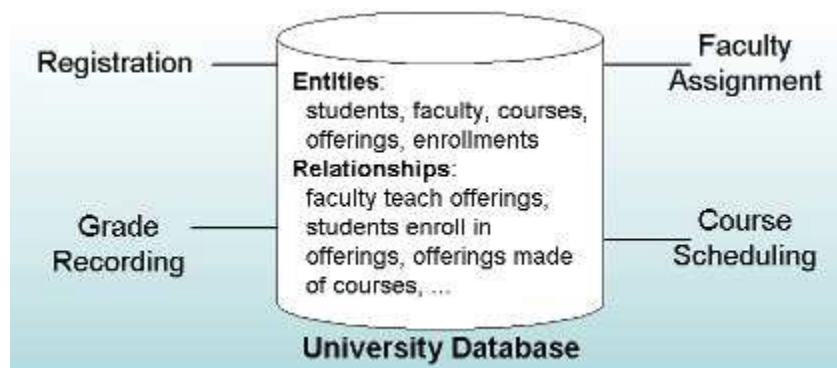
Las principales características de una BD son:

- **Persistencia:** los datos tienen que perdurar en el tiempo. Solamente se tienen que guardar los datos importantes.
- **Relaciones:** las entidades son el conjunto de datos de un tema y entre ellas existen relaciones o conexiones.

- **Compartición:** la base de datos debe ofrecer múltiples usos (diferentes formas de entrar, mantener y visualizar la información) y múltiples accesos (diferentes usuarios deben poder utilizarla al mismo tiempo).

3.2.2. Supuesto de una BD

Dentro de una Universidad queremos llevar el mantenimiento de información diversa (proceso de matrícula, informes de las diferentes calificaciones de los alumnos, programación/horario de las materias y/o cursos, asignación de profesores y alumnos a los diferentes cursos, etc). Cada submundo contiene o comparte diferentes entidades (facultades, cursos, profesores, semestres) y relaciones (los estudiantes realizan asignaturas, los estudiantes estudian en una facultad, la universidad tiene diferentes facultades, cada facultad ofrece una serie de asignaturas,..)



Para definir esta base de datos, debemos especificar la estructura de cada elemento del cuál queremos guardar información indicando los distintos tipos de elementos de información que queremos almacenar en cada elemento. Por ejemplo, en el caso de un estudiante queremos guardar su nombre, DNI, nº de estudiante, las carreras y las asignaturas en que está matriculado. La manipulación de la base de datos consiste en la actualización de los datos y en las consultas que podamos realizar sobre dichos datos.

Ejemplos de consultas:

- Obtener un listado de todos los alumnos que están matriculados en una asignatura concreta
- Qué alumnos obtuvieron más de un 7 en la asignatura de Base de datos el pasado año.
- Cuántos profesores distintos de Base de datos han habido a lo largo de todos los años.

Ejemplos de actualizaciones:

- Modificar la dirección de un alumno.
- Modificar al profesor que imparte una asignatura concreta.
- Borrar una asignatura porque no existe ningún alumno que se haya matriculado.
- Asignar a un alumno en una asignatura.

3.2.3. Metadatos de una BD

Una base de datos es autodescriptiva, en el sentido de que contiene una descripción de sí misma. Por tanto, las bases de datos también almacenan descripciones de los datos que se guardan.

Estos datos de descripción se llaman **metadatos o diccionarios de datos** (datos sobre los datos). La forma y formato de los metadatos varía según el SGBD utilizado.

Un diccionario de datos, según IBM Dictionary of Computing, se puede definir como un “repositorio centralizado de información sobre datos tales como el significado, relaciones con otros datos, origen, uso y formato.”

Los metadatos pueden consultarse para determinar las tablas existentes particulares, los índices u otras estructuras existentes en la base de datos.

3.2.4. Niveles de abstracción en una BD

En cualquier sistema de información (tanto archivos como bases de datos) se considera que se pueden observar los datos desde dos puntos vista:

- **Vista externa.** Esta es la visión de los datos que poseen los usuarios externos.
- **Vista física.** Esta es la forma en la que realmente están almacenados los datos.

En un sistema de archivos, los usuarios ven los datos desde las aplicaciones creadas por los programadores. Esta vista puede ser formularios, informes visuales o en papel... Pero la realidad física de estos datos, tal cual se almacenan en los discos, no la ven. Esta visión está reservada a los administradores.

En el caso de los sistemas de base de datos y según el modelo ANSI1, se añade una tercera vista, que es la vista conceptual. Esta vista se ubica entre la física y la externa. Se habla pues en bases de datos de la utilización de tres esquemas o modelos para representar los datos, entendiendo por modelo las normas que permiten crear esquemas (diseños de la base de datos).

Esquema físico

Se corresponde con la vista del soporte físico informático en cuanto a que se refiere a la forma en que se organizan los datos en el almacenamiento físico (índices o punteros, longitud de los campos, caminos de acceso a los datos, particionamientos de memoria, etc. .) Esta visión la requiere solo el administrador.

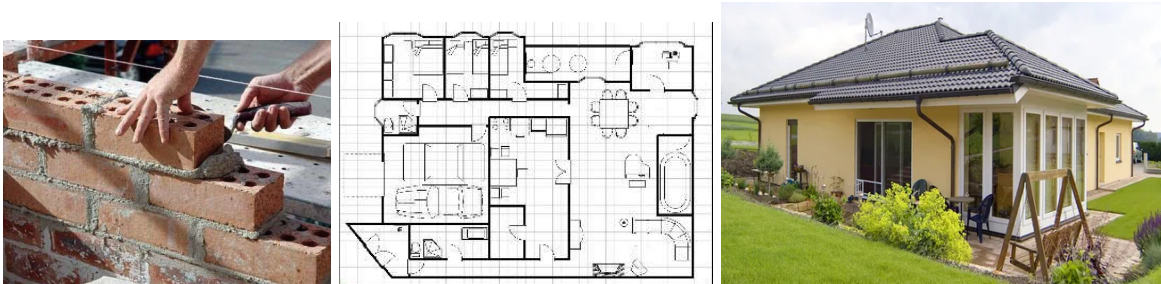
Esquema conceptual

Se corresponde con la visión total de los datos de alto nivel. Es decir, todos los datos y sus relaciones. Esta vista global se interpone entre el nivel externo y el nivel físico, siendo independiente tanto del equipo como de cada usuario en particular.

Se trata de la propuesta teórica de la base de datos y, por tanto, el primer paso para crear una nueva base de datos consiste en el diseño de un modelo para este nivel.

Esquema externo

Se corresponde con la visión de la BD que cada usuario externo tiene en particular. A esa vista individual que cada usuario tiene de la B.D se le denomina vista externa. Esto significa que no todos los usuarios necesitan conocer la BD completa sino que únicamente necesitan una vista parcial de ella (la cual le permita llevar a cabo su trabajo); por ejemplo, un administrativo que trabaje elaborando las nóminas de los empleados de una empresa no necesita conocer los datos relativos a las ventas de productos de esa empresa.



Esquema físico → Esquema conceptual → Esquema externo

3.3. SGDB (en inglés, DBMS DataBase Management System)

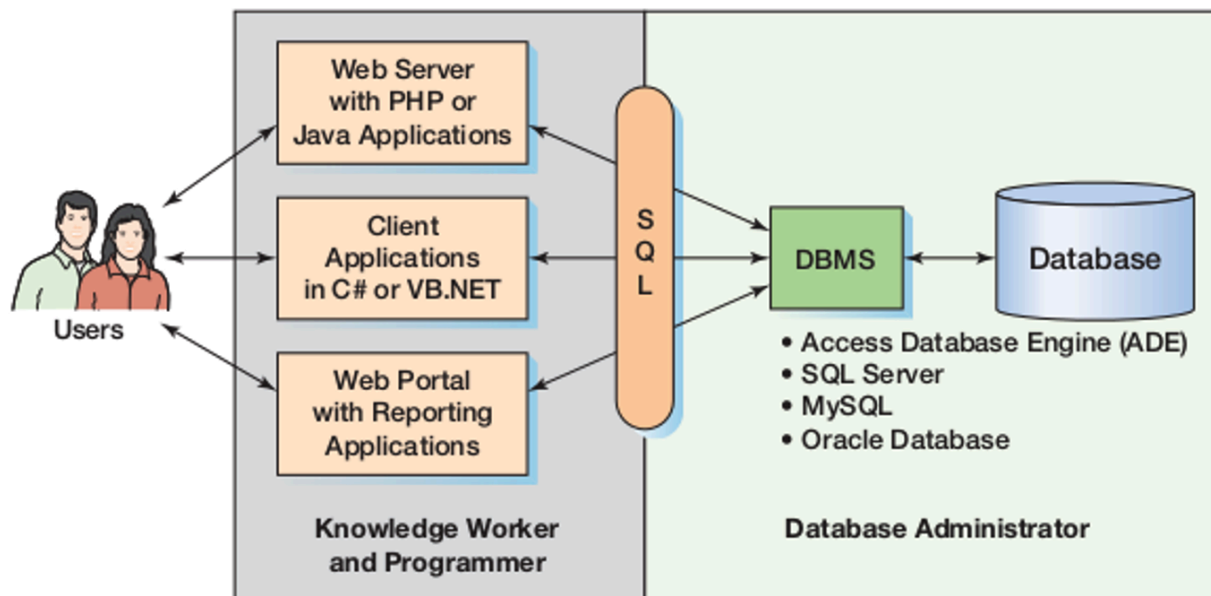
Un Sistema Gestor de Bases de Datos (SGBD) es un conjunto coordinado de programas, procedimientos, lenguajes, herramientas, librerías, etc., que suministra, tanto a los usuarios no informáticos como a los analistas, programadores o administradores de una base

de datos, los medios necesarios para **crear, procesar y administrar los datos almacenados en una base de datos**, garantizando su seguridad.

El éxito del SGBD reside en mantener la seguridad e integridad de los datos.

Ejemplos de SGBD actuales son: Oracle Database, Oracle MySQL, Redis, Elastic Search, Microsoft SQL Server, PostgreSQL, MongoDB... (consultar [ranking actual](#))

La siguiente figura muestra la arquitectura de un sistema de una BD; en ella se observa como el único usuario que trabajará directamente con el SGBD es el administrador. Los programadores de aplicaciones de acceso siempre harán uso de SQL como lenguaje intermedio de peticiones de datos en SGBD y así acceder a la base de datos que contiene toda la información.



3.3.1. Objetivos de los SGDB

Un SGBD debe ofrecer los siguientes servicios:

- **Creación y definición de la BD.** Especificación de la estructura, tipo de datos, restricciones y relaciones entre ellos mediante lenguajes de definición de datos. Toda esta información se corresponde con las tablas y metadatos de la base de datos y el SGBD proporciona mecanismos para su gestión.
- **Manipulación de los datos** realizando consultas, inserciones, actualizaciones o borrados de manera práctica y eficiente.
- **Acceso controlado a los datos de la BD** mediante mecanismos de seguridad

de acceso a los usuarios.

- **Redundancia mínima de los datos** para controlar el espacio físico ocupado por la BD.
- **Integridad y consistencia de los datos** utilizando mecanismos para evitar que los datos sean perjudicados por cambios no autorizados.
- **Acceso compartido a la BD**, controlando la interacción entre usuarios concurrentes.
- **Mecanismos de apoyo, recuperación y copias de seguridad** para restablecer la información en caso de fallos en el sistema, así como la **exportación e importación de datos**.
- **Independencia física y lógica de los datos**. Un cambio sobre el esquema físico no afecta a los esquemas lógicos ni externos. Un cambio sobre el esquema lógico no afecta ni a los físicos ni a los externos.

3.3.2. Elección del SGDB

El SGDB a escoger irá en función de los requisitos del proyecto y el soporte que deseemos tener.

Principalmente tendremos:

- **SGBD propietarios** (pago por licencia: Oracle, MS SQL Server...)
- **SGBD libres** (PostgreSQL, MariaDB...)



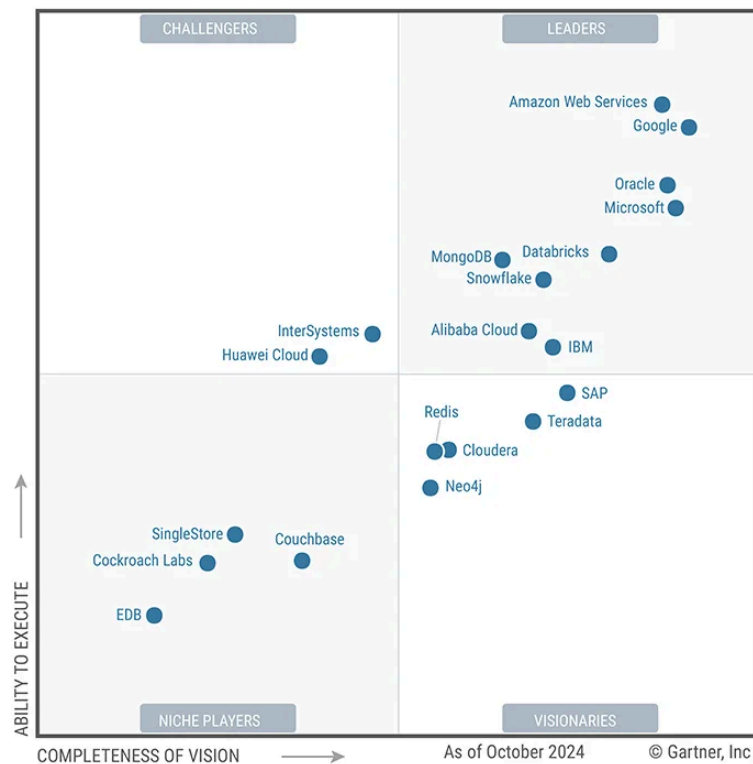
Cualquiera de los dos acceden o se puede presentar en modalidad **Cloud (Nube/Remoto)** o **On Premise (Local)**, y algunas como AWS o Snowflake solamente se pueden ser contratadas en Cloud.

Un buen indicador de cómo está el mercado de las nuevas tecnologías es ver el famoso **cuadrante mágico** de la consultora americana **Gartner**, destinada a la consultoría e investigación en el mercado de las nuevas tecnologías.

Agrupar a los competidores por:

- **Líderes (leaders):** Son quienes tienen la mejor puntuación resultante de combinar la habilidad de ejecución y el alcance de visión.
- **Aspirantes (challengers):** Caracterizados para ofrecer buenas funcionalidades y un número considerable de instalaciones del producto, pero sin la visión de los líderes.
- **Visionarios (visionaries):** tienen habilidad para anticiparse a las necesidades del mercado, pero no responden con una plataforma sólida para responder a las necesidades a nivel global.
- **Nichos específicos (niche players):** enfocados a determinadas áreas de las tecnologías de almacenamiento de BD, pero sin innovación.

Figure 1: Magic Quadrant for Cloud Database Management Systems



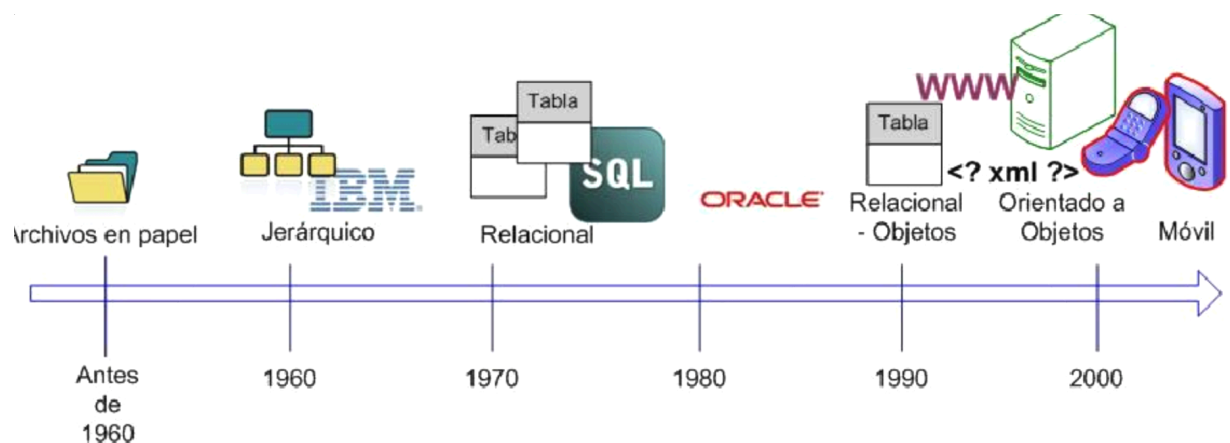
Gartner.

3.4. SQL

SQL, Structured Query Language, es un lenguaje estandarizado por la ISO (International Organization for Standardization), es decir, que todos los SGBD que lo soportan deben tener la misma sintaxis a la hora de aplicar el lenguaje. Se divide en 4 sublenguajes, el conjunto de los mismos permite al SGBD cumplir con sus funcionalidades:

- **Lenguaje DDL:** lenguaje de definición de datos (Data Definition Language). Permite crear toda la estructura de una base de datos (desde tablas hasta usuarios o roles), las relaciones entre los datos y las reglas que deben cumplir. Sus cláusulas son de tipo DROP (borrar), ALTER (modificar) y CREATE (crear).
- **Lenguaje DML:** lenguaje de manipulación de datos (Data Manipulation Language). Este lenguaje permite: SELECT (seleccionar/buscar) , INSERT (insertar), UPDATE (actualizar) y DELETE (borrar). Si la función sólo se reduce a buscar datos, utiliza el término **DQL** (Data Query Language).
- **Lenguaje DCL:** lenguaje de control de datos (Data Control Language). Incluye acciones como GRANT y REVOKE que permiten al administrador gestionar el acceso a los datos contenidos en la base de datos.
- **Lenguaje TCL:** lenguaje de control de transacciones (Transaction Control Language). El propósito de este lenguaje es permitir ejecutar varias acciones de forma simultánea como si fueran una acción atómica o indivisible. Si es posible, se aplica la transacción (COMMIT), y si, en algún paso de la ejecución ocurre algo inesperado, se pueden deshacer todos los pasos realizados (ROLLBACK).

4. HISTORIA DE LAS BASES DE DATOS



[Vídeo introductorio](#)

La historia de las bases de datos comienza en Estados Unidos en el año 1963 cuando se utilizó este término por primera vez para referirse al proceso automático o pseudo-automático de almacenamiento organizado de datos. No obstante, siglos antes ya se aplicaba el concepto de base de datos para referirse a la recolección de datos de forma manual.

4.1. Década de los años 50: Ficheros

Una aplicación trabajaba con pocos archivos para generar un nuevo archivo (procesos batch o por lotes). Presenta problemas de integridad, acceso concurrente, seguridad; así como poca flexibilidad (era necesario reescribir las aplicaciones si había cambios en los datos, en las consultas...). Se trabajaba con cintas magnéticas que se leían de forma secuencial.

4.2. Década de los años 60: BD Jerárquicas y en Red

En 1961 [Charles Bachman](#) diseñó el primer SGBD generalizado: el almacén de datos integrados (Integrated Data Store, IDS) de General Electric. La idea fundacional de las Base de Datos (en realidad, los SGBD) era solucionar todos los problemas derivados del uso aislado de distintos archivos.

Características:

- Sistemas centralizados: gran ordenador (mainframes) y terminales “tontos”
- Aparecieron las primeras bases de datos basadas en el Modelo de datos jerárquico y en red.
 - Las BD jerárquicas almacenan la información en una estructura jerárquica que podemos imaginar con una forma de árbol invertido, donde cada nodo padre puede tener distintos hijos. El nodo superior, que no tiene padre, se conoce como raíz. Y a los nodos que no tienen hijos se les llama hojas.
 - Las BD en red (similar al modelo jerárquico) con registros interrelacionados mediante una estructura en forma de árbol invertido, pero más flexible, ya que permitía que los nodos tuvieran más de un solo padre.

4.3. Década de los años 70: Modelo Relacional y DBMS

4.3.1. Años 1970-72

[Edgar Codd](#) (IBM) propone el Modelo de Datos Relacionales. Se basa en la lógica de predicados y en la teoría de conjuntos (áreas de la lógica y de las matemáticas). Actualmente,

es el sistema más ampliamente utilizado para modelizar datos. Codd definió un conjunto de 13 reglas/leyes (0-12) para establecer las características que debe tener una base de datos relacional. **Actualmente, todos los SGBD implementan estas reglas.** Durante esta década y gracias a Codd se crearon los dos SGBD: Relational Software System y System R, creado por IBM y utilizando el lenguaje de consultas SEQUEL siendo el precursor del lenguaje SQL.

4.3.2. Año 1976

El doctor [Peter Chen](#) propuso un modelo conceptual llamado Entidad-Relación (Entity-Relationship/ER) para diseñar Bases de Datos.

4.3.2. Segunda mitad de los años 70

El software de gestión de archivos era demasiado sencillo:

- El tratamiento de interrelaciones no estaba soportado.
- Varios usuarios no podían actualizar datos a la vez.

Durante la 2ª mitad de los años 70 surge un software más sofisticado: los Data Base Management System (DBMS) o Sistemas Gestores de Bases de Datos (SGBD) que permitían resolver estas limitaciones.

Nace Oracle, el SGBD más importante en la actualidad.

4.4. Década de los años 80

- Se consolida el modelo relacional como la base teórica de los SGBD Comerciales.
- Durante la década de los 80 IBM lanza su motor de base de datos DB2 basándose en su System R.
- Aunque el sistema relacional se inventó en la década anterior, es en ésta donde coge más vuelo y se postula como principal modelo de datos para muchas empresas utilizando los SGBD relacionales.
- Unos años más tarde, IBM crea el lenguaje **SQL** (Structured Query Language), un potente lenguaje de consultas para manipular la información de base de datos relacionales.
- En 1986, el Instituto Nacional Norte americano de Normalización (American National Standards Institute, o ANSI, en inglés) publicó las primeras normas que enunciaban la sintaxis y la semántica del SQL y éste se convierte en el lenguaje estándar de las BD relacionales.

4.5. Década de los años 90

- Los fabricantes de SGBD consolidan sus productos y proporcionan extensiones por el soporte de nuevos tipos de datos (multimedia,...) SGBD son el pilar de muchos sistemas Cliente / Servidor a través de Internet.
- Aparecen los primeros prototipos de **SGBD orientados a objetos** (Object Database Management Systems, ODBMS, XML, documentales, Datawarehouse, Geolocalización...
- A mediados del 90s, IBM saca una versión de DB2 capaz de dividir una base de datos enorme en varios servidores comunicados por líneas de gran velocidad, creando así las **bases de datos paralelas**.
- A finales de década IBM y Oracle incorporan a sus BD la capacidad de manipular objetos, creando así las bases de datos orientadas a objetos. Estas bases de datos orientadas a objetos se basan en la existencia de objetos persistentes que se almacenan para su procesamiento mediante programas orientados a objetos. En vez de almacenar relaciones y tablas.
- Finalmente, durante los años noventa, la implantación en las BD, incluso en pequeños sistemas personales, motivó la aparición de los llamados lenguajes de cuarta generación (fourth generation languages, o 4GL, en inglés) específicos para Bases de Datos.

4.6. Actualidad

Aunque a día de hoy los sistemas relacionales son todavía los que predominan en el mercado y la gran mayoría de aplicaciones los utilizan, a partir del año 2000 y, a causa del BigData, ha tomado fuerza un nuevo tipo de base de datos llamadas **NoSQL** (MongoDB, Hadoop/ Hbase, Cassandra, CouchDB, Riak...). Ventajas:

- Capacidad de tratar grandes volúmenes de datos de forma óptima
- Escalabilidad
- Ejemplos de empresas que las utilizan: Amazon, Google, Meta...

Otro paradigma que ha aparecido recientemente es el **cloud computing** , donde no debemos hacer inversiones en la adquisición de SGBD y se realiza un pago por uso.

4.6.1. Panorama actual de las bases de datos NoSQL

El término NoSQL sobre el año 1998 se utilizaba para referenciar bases de datos relacionales de código abierto que no utilizaban lenguaje de consulta SQL. A partir de 2009 se

utiliza este término para **bases de datos distribuidas de código abierto no relacionales**.

Dentro de todas las implementaciones actuales de base de datos NoSQL se encuentran algunas que no utilizan SQL (como MongoDB que utiliza JSON), pero sí que hay otras que lo siguen utilizando, como BigTable (GQL).

El término **NoSQL** en bases de datos intenta resolver las limitaciones de los modelos relacionales en cuestión de almacenamiento masivo de datos donde se necesita tener servidores muy potentes y con carga balanceada.



- **MongoDB es el DBMS NoSQL líder** y permite a empresas ser más ágiles y escalables.
- Ha sido creada para brindar mayor rendimiento y gran disponibilidad.
- Está orientada a documentos, que son almacenados en BSON, que es una representación binaria de JSON.
- Es utilizado por compañías como SourceForge, Foursquare, MTV, Disney, IGN, Guardian, NYTimes, Doodle.



- Hadoop es un framework que permite el procesamiento de grandes volúmenes de datos a través de clusters.
- Es un sistema distribuido que utiliza una arquitectura Master Slave, usando para almacenar su Hadoop Distributed File System (HDFS) y algoritmos de MapReduce para realizar cálculos.
- **HBase es la base de datos de Hadoop**, que no admite SQL y no sigue el esquema relacional. Lo utilizan Amazon, Adobe, AOL, Ebay, Facebook, IBM, Microsoft, Twitter, Yahoo.

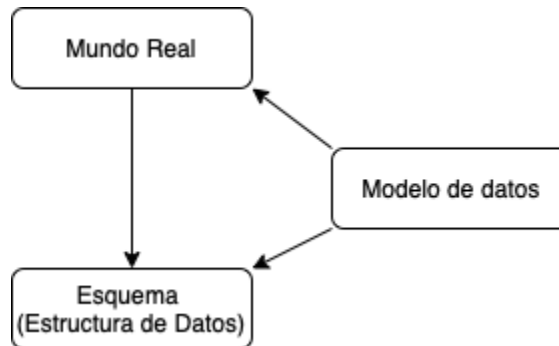
5. MODELOS DE DATOS COMO INSTRUMENTO DE DISEÑO DE BASES DE DATOS

5.1. ¿Qué es un modelo de datos?

Una BD es una representación de la realidad. Es decir, un modelo de realidad. Nosotros queremos transformar la realidad y plasmarla en un entorno en el que la podamos tratar,

manipular y gestionar.

El diseño de la BD consiste en describir la estructura de la BD de forma que se represente fielmente la parcela del mundo real (en el ámbito de las bases de datos se denomina Universo del Discurso (UD)) que se desea almacenar. Esto se realiza mediante un proceso de abstracción (que se denomina **modelado**) que se apoya en un modelo de datos. **Un modelo de datos es el instrumento que se aplica a un UD para obtener una estructura de datos que se denomina esquema de la BD.**



Un **modelo de datos** proporciona un **conjunto de conceptos, reglas y convenciones** que nos permiten especificar y manipular los datos que deseamos almacenar en la BD. Todo modelo de datos se compone por una parte estática y de una parte dinámica como se explica a continuación:

- **Estática:** Conjunto de estructuras (también denominados constructores del modelo) que permiten definir los datos y sus restricciones asociadas especificadas según un Lenguaje de Definición de Datos. Esta parte estática consta de elementos permitidos y elementos no permitidos:
 - **Elementos permitidos:** son los objetos, asociaciones entre objetos, propiedades, etc. que proporciona el modelo para representar una determinada realidad (estos elementos varían de un modelo a otro).
 - **Elementos no permitidos:** Son las restricciones que representan las limitaciones impuestas a la estructura del modelo o a los datos que invalidan ciertos ejemplares de la BD. Las restricciones son de dos tipos:
 - **Restricciones inherentes:** son las limitaciones impuestas a las estructuras del modelo (reglas impuestas para la utilización y combinación de los distintos constructores del modelo).
 - **Restricciones semánticas** (o de usuario): son las restricciones que se deducen de los supuestos semánticos explícitos o implícitos o derivados de nuestro conocimiento del mundo real que se quiere reflejar en la BD

(por ejemplo, "todo empleado debe pertenecer a un departamento ", "el sueldo de un determinado empleado siempre será inferior al sueldo de su jefe", etc.

- **Dinámica:** Está formada por un conjunto de operadores que permiten manipular los datos y que están reflejados en el Lenguaje de Manipulación de Datos (DML). La parte dinámica no tiene sentido para todos los modelos de datos.

Además, cada modelo de datos tiene una representación gráfica que suele ser en forma de grafos o tablas.

5.2. Modelos de datos más utilizados

Los modelos de datos más utilizados a lo largo del tiempo han sido los siguientes, expuestos en orden de aparición:

- **Modelo Jerárquico:** fue el primer modelo de BD utilizado. Apareció a principios de los años sesenta. Sus estructuras son registros interrelacionados en forma de árboles. El SGBD clásico de este modelo es el IMS/DL1 de IBM. El modelo jerárquico fue desarrollado para permitir la representación de situaciones de la vida real en las que predominan las relaciones de tipo 1:N (Un curso(1) hay diferentes alumnos(N))
- **Modelo en Red:** El modelo de datos en red representa a las entidades en forma de nodos de un grafo y las relaciones entre éstas mediante arcos que unen los diferentes nodos. Con este modelo se mejoraba el anterior de tal forma que se daba una solución más eficiente al problema de redundancia de datos, pero aún así la dificultad de administrar la información de una base de datos en red ha significado que sus usuarios sean puramente técnicos.
- **Modelo Relacional:** tal y como hemos dicho anteriormente, este modelo fue introducido por Codd (1970). Así como los modelos prerelacionales (jerárquico y en red) las estructuras de datos constan de dos elementos básicos: los registros y las interrelaciones; en el modelo relacional la forma informal de un solo elemento: la tabla (formada por filas y columnas). Las interrelaciones deben modelizarse utilizando las tablas.
- **Modelo Orientado a Objetos:** en el año 2000 se extendió el modelo de BD relacional con objetos. Se trata de ampliar el modelo relacional, añadiendo la posibilidad de que los tipos de datos sean tipos abstractos de datos (TAD).
- Debido al boom de internet y los grandes volúmenes de datos, han entrado en juego los SGBD llamados **noSQL** con los siguientes puntos característicos: no relacionales, distribuidos, schemaless (sin esquema fijo), open source (algunos) y fácilmente escalables.

5.3. Modelo Relacional vs NoSQL

¿Cuándo utilizar Relacional?

- Trabajamos con una base de datos de tamaño medio (10-100GB) con poca concurrencia (cientos de usuarios como máximo).
- Cuando queremos almacenar una amplia variedad de datos en cientos o miles de tablas normalizadas.
- Cuando necesitamos garantizar transacciones seguras (**ACID** (Atomicity, Consistency, Isolation, Durability), especialmente Atomicidad y Consistencia) y los datos tienen muchas relaciones entre ellos (jerarquías, relaciones maestro-esclavo, etc).
- Los sistemas relacionales están diseñados para ser escalados verticalmente y desde el principio necesitan máquinas potentes con mucha cantidad de memoria, rápido I/O throughput mediante SANs y SSDs. Se escalan horizontalmente ocasionalmente a través de clustering y partición de datos.

¿Cuándo utilizar NoSQL?

- Queremos implementar a gran escala una base de datos de alta concurrencia (cientos de GB con miles de usuarios).
- No requerimos de las reglas ACID.
- No requerimos de relaciones y restricciones en el esquema de datos.
- Utilizamos pocas tablas (5-10 tablas en el modelo relacional).
- Queremos implementar alta disponibilidad con costes bajos.

Estos sistemas son buenos en muchos de los sitios web (websites), como por ejemplo, Google o X. ¿Creéis que es realmente importante si se retrasan unos segundos algunos tweets o incluso que se pierdan? ¿Y si Google no muestra la información de resultados de la última página web que se ha creado o que él ha indexado? O, por ejemplo, ¿si los resultados de Google pierden algún enlace por el camino?

6. USUARIOS DE BD

Podemos clasificar los diferentes usuarios que interaccionan con las bases de datos de la siguiente manera:

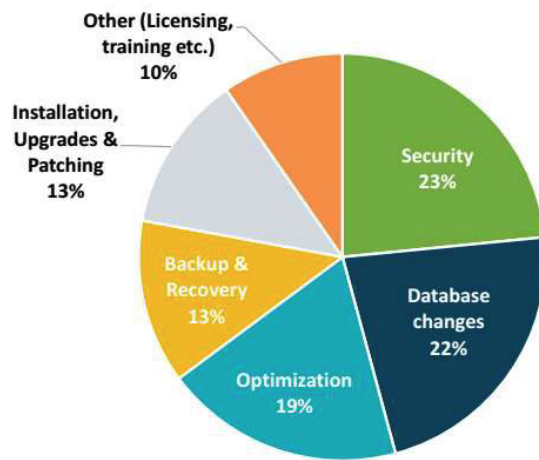
- **DBAs** (DataBase Administrator, Administradores de Bases de Datos): realizan tareas de administración y control centralizando los datos y gestionando los permisos de acceso concedidos a los diferentes usuarios y grupos de usuarios, con tal de garantizar el

funcionamiento correcto de la BD. (rol principal de un alumno de ASIR)

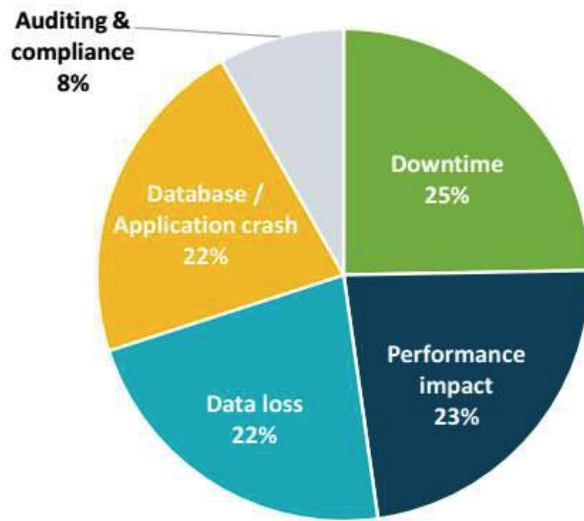
- **Diseñadores de Bases de Datos:** encargados de diseñar el esquema de una BD, identificar los datos que se almacenan en la BD y escoger las estructuras adecuadas para representar y almacenar estos datos. (rol principal de un alumno de DAW/DAM)
- **Usuarios finales:**
 - **Usuarios especializados:** interactúan directamente con el sistema, sin utilizar las interfaces proporcionadas por programas intermedios. Formulan consultas en un lenguaje de BD (normalmente BD).
 - **Usuarios de aplicaciones:** son usuarios que no interactúan directamente con el sistema, sino que mediante alguna aplicación informática desarrollada previamente por otras personas con esta finalidad.

6.1. Tareas de los DBAs

¿En qué invierten su tiempo los DBAs? (según una encuesta del 2018 de Database DevOps Survey de DBmaestro)



¿Cuáles son los riesgos más habituales cuando despliegan cambios en las BDs según dicha encuesta?



7. BASES DE DATOS DISTRIBUIDAS

Un **sistema de computación distribuida** consiste en un conjunto de computadores (que no necesariamente deben ser homogéneos), que están interconectados entre sí formando una red, y que cooperan para realizar una determinada tarea. Un sistema de computación distribuida parte un problema grande en pequeñas piezas, y soluciona cada una de ellas eficientemente de forma coordinada.

Podemos definir una **base de datos distribuida** (BDD o DDB, Distributed Database System) como aquella cuyos datos están repartidos entre más de una máquina, y un **sistema de gestión de bases de datos distribuido** (SGBD o DDBMS, Distributed Database Management System) como el software que gestiona una base de datos distribuida haciendo que la distribución de los datos sea transparente al usuario, es decir, los usuarios actúan como si todo estuviera junto en una sola base de datos.

Por muy dimensionado que tengamos un servidor (memoria, CPUs, disco duros) si vamos añadiendo usuarios llegará un momento en que por **limitaciones físicas** no podrá dar servicio. En entornos de internet donde no podemos controlar el número de usuarios que se conectarán a nuestro sistema es muy importante poder **distribuir la carga**.

Además de las funciones de un SGBD centralizado, un **SGBDD debe tener las siguientes funciones**:

- Servicios de comunicación extendidos para proporcionar acceso a localizaciones remotas y permitir la transferencia de consultas y datos entre las localizaciones usando

una red.

- Catálogo del sistema extendido para almacenar detalles sobre la distribución de los datos.
- Procesamiento distribuido de consultas, incluyendo la optimización de consultas y acceso a datos remotos.
- Control de concurrencia extendido para mantener la consistencia de los datos replicados.
- Servicios de recuperación extendidos que tengan en cuenta fallos en las localizaciones individuales y fallos de conexiones de comunicación.

El principio fundamental de las bases de datos distribuidas es el siguiente:

***"Desde el punto de vista del usuario, un sistema distribuido
deberá ser idéntico a un sistema no distribuido."***

Esta regla fundamental da lugar a las siguientes doce reglas:

1. Autonomía local (grado de independencia de cada SGBD individual)
2. No dependencia a un lugar central
3. Operación continua
4. Independencia respecto a la localización
5. Independencia respecto a la fragmentación
6. Independencia de réplica
7. Procesamiento distribuido de consultas
8. Manejo distribuido de transacciones
9. Independencia respecto al equipo
10. Independencia respecto al sistema operativo
11. Independencia respecto a la red
12. Independencia respecto al SGBD

7.1. Ventajas de los SGBDD

- Compartición de datos
- Autonomía
- Disponibilidad
- Crecimiento modular

7.2. Inconvenientes de los SGBDD

- Coste de desarrollo del software

- Rendimiento
- Integridad
- Optimización
- Compatibilidad de datos

7.3. CAP Theorem

El CAP theorem, también conocido como teorema de Brewer, establece que es imposible que un sistema informático distribuido proporcione simultáneamente las tres garantías siguientes:

- Consistencia (todos los nodos ven los mismos datos al mismo tiempo)
- Disponibilidad (una garantía de que cada solicitud reciba una respuesta sobre si fue exitosa o fallida)
- Tolerancia a particiones (el sistema continúa funcionando a pesar de la pérdida arbitraria de mensajes)

Más información sobre el CAP Theorem:

- http://en.wikipedia.org/wiki/CAP_theorem
- <http://dbmsmusings.blogspot.com/2010/04/problems-with-cap-and-yahoos-little.html>

7.4. Supuesto de aplicación distribuida

Por ejemplo, imaginemos que tenemos una aplicación distribuida de reservas de apartamentos. Para mejorar el rendimiento tenemos la base de datos distribuida en dos continentes: Europa y América.

Imaginemos que la comunicación entre Europa y América falla. ¿Qué hacemos con los clientes que desean hacer una reserva en el mismo apartamento? ¿Les dejamos hacer reservas o no? ¿Qué favoremos?

- Consistencia: No dejamos realizar la reserva, porque no sabemos si Europa o América tienen una reserva para ese apartamento concreto.
- Disponibilidad: Dejamos realizar la reserva y ya arreglaremos los posibles problemas de consistencia más adelante.

Importante: Será el negocio/cliente quién decidirá qué quiere hacer!

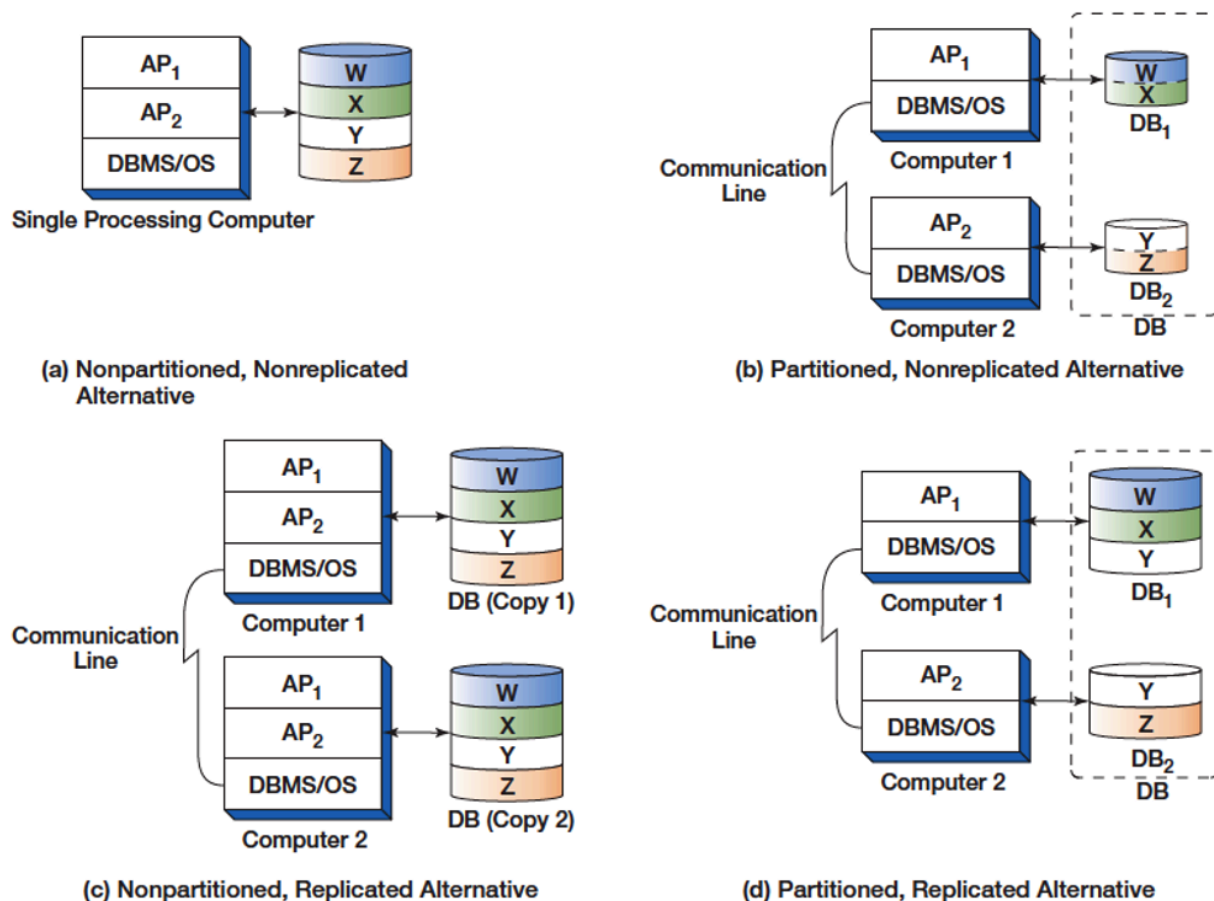
7.5. Técnicas para la creación de BD distribuidas

Cuando queremos distribuir una base de datos en múltiples nodos, diremos que

estamos haciendo un **escalado horizontal** (scalling-out) y se podrá realizar mediante las siguientes técnicas:

- **Replicación:** almacenaje de copias de la bd en múltiples ordenadores (redundancia intencionada). Facilita mucho las operaciones de lectura, pero supone operaciones de actualización complejas (peligro de perder la consistencia de los datos) Proporciona un alto rendimiento con un failover automatizado. Sus subtipos son:
 - **Replicación master-master** (también conocido como Multi-Master o Cluster Homogéneo)
 - **Replicación master-slave** (también denominado NDB)
- **Particionamiento o fragmentación:** división de la base de datos en partes y su almacenamiento en múltiples computadores. Consiste en proporcionar particionado de grandes datasets sobre diversas máquinas de **forma transparente**. Su objetivo es lograr la funcionalidad de load balancing (equilibrio de carga).

Estas dos técnicas pueden aplicarse de forma individual (imágenes b y c), aunque habitualmente las encontramos combinadas (imagen d):



Pregunta: ¿Por qué crees que aparecieron las CDN?

8. BIG DATA

Se llama Big Data(datos masivos o macrodatos) a los conjuntos de información cuyo volumen sobrepasa las capacidades de las herramientas convencionales de procesamiento de datos. También se refiere a la recopilación de todos estos datos y nuestra capacidad de utilizarlos a nuestro favor en una gran diversidad de áreas, desde los negocios hasta la política.

Las 4 Vs del BigData:

- **Volumen:** grandes cantidades de datos a tratar. Órdenes de magnitud de Petabyte-Zettabyte
- **Variedad:** datos estructurados, no estructurados, textos, imágenes, etc...
- **Velocidad:** cantidad de tiempo que invertimos en el procesado de los datos.
- **Veracidad:** fiabilidad de la información que obtenemos.

Podemos añadir otras 'v' como:

- **Variabilidad:** capacidad de los datos de cambiar constantemente. El significado de una palabra puede variar dependiendo del contexto en el que nos encontramos.
- **Valor:** convertir los datos brutos en información útil (de todo lo que hacemos debemos conseguir un valor).
- **Visualización:** es necesario un mecanismo de facilidad de lectura y comprensión.

Consulta la web de Worldometer

8.1. Conceptos básicos

- ❖ **Procesamiento distribuido:** los datos se procesan en distintas máquinas (nodos) consiguiendo mejorar la velocidad. No trabajar con un solo archivo, sino dividir los datos en diferentes máquinas y que estas procesen su parcela de datos.
- ❖ **Escalabilidad horizontal:** los sistemas crecen muy rápido. Por tanto, necesitamos sistemas que nos permitan añadir nuevos nodos y no añadir nuevo HW a las máquinas existentes. Con esto conseguiremos que mediante hardware económico podamos realizar el trabajo sin necesidad de realizar grandes inversiones.
- ❖ **Tolerancia a fallos:** es necesario que la infraestructura sea tolerante a fallos. Si un nodo o máquina cae, el sistema debe seguir funcionando y redistribuir el trabajo de las

máquinas que han caído.

- ❖ **Ubicación de los datos:** cada nodo/máquina trabajará con los datos que tenga más cerca. Así conseguimos evitar cuellos de botella y minimizar latencias.

8.2. Fases del ciclo de vida

1. **Ingesta:** los datos procedentes de una fuente externa (web, BBDD,...) entran en nuestro sistema para empezar a trabajar con ellos. Por ejemplo, un proceso nocturno que recoja los datos de la BBDD y los trate o un proceso en streaming que vaya cogiendo tweets o datos IoT.
2. **Persistencia:** es necesario guardar los datos y almacenarlos en nuestro sistema. Puede ser en una BD relacional o no relacional.
3. **Procesamiento:** etapa en la que procesamos y analizamos los datos realizando operaciones para conseguir valor.
4. **Visualización:** mostrar gráficamente (o de una forma amigable) los resultados obtenidos por aquellas personas que extraerán conclusiones.

8.3. Casos de uso

- **Procesado de logs:** análisis de grandes cantidades de datos desestructurados que se convierten en valor. Por ejemplo, datos de los consumos energéticos de una vivienda, procesamiento de logs de servidores para encontrar cuellos de botella, en los que hay más errores, etc...
- **Sistemas de recomendación:** recomendación de productos gracias al análisis de las preferencias y perfiles de otros usuarios
- **Buscadores web:** búsqueda y ordenación de millones de páginas de acuerdo a unas palabras clave.
- **Salud:** procesado de análisis clínicos, ADN, hábitos alimentarios y ayuda a comprender enfermedades. Si controlamos qué pacientes con ciertos hábitos han sufrido ciertas enfermedades, podremos sacar conclusiones.
- **Deporte:** estudiar y analizar estadísticamente y vídeos para sacar patrones típicos de juego, analizar rivales, etc... (película [Moneyball](#))
- **Mundo financiero:** cuándo comprar/vender, análisis de mercado, etc ...